EE-170L Computer Programming Lab

Laboratory Report — No. 03

Relational Operators, Logical Operators & Conditional Statements in C++

Student Name	Zawar Shah
Registration No.	BF25NWELE0677
Course	EE-170L – Computer Programming Lab
Lab No.	03
Topic	Relational & Logical Operators, Conditional Statements (if / if-else / else-if)
Semester	Spring 2026
Institution	UET Peshawar

1. Lab Objectives

By the end of this lab session, students will be able to:

- Understand and apply Comparison / Relational Operators in C++.
- Understand and apply Logical Operators (AND, OR, NOT) in C++.
- Write programs using the if statement for single-condition branching.
- Write programs using the if-else statement for two-way decision making.
- Write programs using the else-if ladder for multi-way decision making.

2. Relational Operators in C++

Relational operators are used to compare two values. The result of any relational expression is either true (1) or false (0). The table below lists all relational operators available in C++:

C++ Symbol	Math Notation	Meaning
<	<	Less than
>	>	Greater than
<=	≤	Less than or equal to
>=	≥	Greater than or equal to
==	=	Equal to
!=	≠	Not equal to

Example — Voting Eligibility (using >= relational operator):

```
#include <iostream>
using namespace std;
```

```
int main() {
    int age;
    cout << "Enter your age: ";
    cin >> age;

    if (age >= 18) {
        cout << "You are eligible to vote." << endl;
    }
    else {
        cout << "You are not eligible to vote." << endl;
    }
    return 0;
}
```

3. Logical Operators in C++

Logical operators are used to combine or invert boolean expressions. They allow a program to evaluate multiple conditions simultaneously:

C++ Symbol	Syntax / Meaning	Example
!	Logical negation (NOT)	!a
&&	Logical AND	a && b
||	Logical OR	a || b

Example — Entry Check (using && logical AND operator):

```
#include <iostream>
using namespace std;

int main() {
    int  age;
    char hasID;
    cout << "Enter your age: ";
    cin  >> age;
    cout << "Do you have an ID card? (y/n): ";
    cin  >> hasID;

    // Using logical AND (&&)
    if (age >= 18 && hasID == 'y') {
        cout << "You can enter." << endl;
    }
    else {
        cout << "Entry not allowed." << endl;
    }
    return 0;
}
```

4. Decision Control Structures in C++

In C++, instructions are executed sequentially by default. Decision control structures allow certain blocks of code to execute only when specific conditions are met. Three main forms are covered in this lab:

- The if statement
- The if-else statement
- The else-if ladder

4.1 The if Statement

The if statement executes its body only when the given condition evaluates to true. If the condition is false, execution continues to the next statement after the block.

Syntax:

```
if (expression)
{
    // Block of statements executed when expression is true
}
```

Example — Number Greater Than 100:

```
#include <iostream>
using namespace std;

int main() {
    int number;
    cout << "Enter a number: ";
    cin  >> number;

    if (number > 100) {
        cout << "The number is greater than hundred" << endl;
    }
    return 0;
}
```

4.2 The if-else Statement

The if-else statement provides a two-way branch. If the condition is true the if-block executes; otherwise the else-block executes. Exactly one of the two blocks always runs.

Syntax:

```
if (expression)
{
    // Block executed when expression is true
}
else
{
    // Block executed when expression is false
}
```

Example — Subtract Larger from Smaller:

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter first integer number : \n";
```

```
    cin  >> a;
    cout << "Enter second integer number : \n";
    cin  >> b;

    if (a > b)
        cout << a << " - " << b << " = " << a - b;
    else
        cout << b << " - " << a << " = " << b - a;

    return 0;
}
```

4.3 The else-if Statement

The else-if ladder is used for multi-way decisions. Conditions are evaluated from top to bottom; as soon as one evaluates to true, its block executes and the rest are skipped. A final else acts as a default catch-all.

Syntax:

```
if (expression)
{
    // Block 1
}
else if (expression)
{
    // Block 2
}
else if (expression)
{
    // Block 3
}
// ... as many else-if blocks as needed ...
else
{
    // Default block
}
```

5. Lab Tasks & Solutions

Task 1 — Relational Operators: Compare Two Numbers

Prompt the user to enter two numbers. Use comparison operators to determine whether the first number is equal to, greater than, or less than the second, and display the result.

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter first number : ";
    cin  >> a;
    cout << "Enter second number: ";
    cin  >> b;

    if (a == b)
        cout << "Both numbers are EQUAL." << endl;
    else if (a > b)
        cout << "First number is GREATER than second." << endl;
    else
```

```
        cout << "First number is LESS than second." << endl;

    return 0;
}
```

Task 2 — Logical Operators: Number in Range 1–99

Prompt the user to enter an integer. Use logical operators to check whether the number is greater than 0 AND less than 100. Display an appropriate message.

```
#include <iostream>
using namespace std;

int main() {
    int num;
    cout << "Enter an integer: ";
    cin  >> num;

    if (num > 0 && num < 100)
        cout << "The number is in range (1 - 99)." << endl;
    else
        cout << "The number is NOT in range (1 - 99)." << endl;

    return 0;
}
```

Task 3 — if Statement: Check Positive Number

Prompt the user to enter an integer. Use an if statement to determine whether the number is positive and display the result.

```
#include <iostream>
using namespace std;

int main() {
    int num;
    cout << "Enter an integer: ";
    cin  >> num;

    if (num > 0)
        cout << "The number is POSITIVE." << endl;
    else
        cout << "The number is NOT positive." << endl;

    return 0;
}
```

Task 4 — if-else Statement: Voting Eligibility by Age

Prompt the user to enter their age. Use an if-else statement to check whether the age is 18 or above and display an appropriate message for each case.

```
#include <iostream>
using namespace std;

int main() {
    int age;
```

```
    cout << "Enter your age: ";
    cin  >> age;

    if (age >= 18)
        cout << "You are eligible to vote." << endl;
    else
        cout << "You are not eligible to vote." << endl;

    return 0;
}
```

Task 5 — else-if Statement: Positive, Negative, or Zero

Prompt the user to enter an integer. Use an else-if ladder to determine whether the number is positive, negative, or zero, and display the appropriate message.

```
#include <iostream>
using namespace std;

int main() {
    int num;
    cout << "Enter an integer: ";
    cin  >> num;

    if (num > 0)
        cout << "The number is POSITIVE." << endl;
    else if (num < 0)
        cout << "The number is NEGATIVE." << endl;
    else
        cout << "The number is ZERO." << endl;

    return 0;
}
```

Task 6 — Additional Task Sheet

A separate task sheet will be provided by the instructor for further practice exercises covering the concepts introduced in this lab.

6. Conclusion

This lab provided a thorough, hands-on introduction to conditional programming in C++. The following key concepts were studied and applied:

- **Relational Operators:** Six operators (<, >, <=, >=, ==, !=) used to compare values and produce boolean results.
- **Logical Operators:** Three operators (!, &&, ||) used to combine or invert boolean conditions.
- **if Statement:** Executes a block of code only when a single condition is true.
- **if-else Statement:** Provides a two-way branch — one block for true, another for false.
- **else-if Ladder:** Enables multi-way decisions by chaining multiple conditions sequentially.

Mastering these decision-making constructs is essential for building programs that respond intelligently to different inputs and situations, forming the foundation for more complex control flow structures covered in future labs.